Hands-On Generative AI

Agentic AI

Day 2 - Session 4

KavinSchool

What We'll Cover

1. Tool use and function calling
2. Agentic AI: multi-step reasoning and the ReAct pattern
3. Frameworks and the Model Context Protocol (MCP)
4. Two local demos: the function-calling loop, a ReAct agent
5. Lab 2.4: function calling and a simple agent
6. Day 2 wrap-up

Tool Use and Function Calling

Giving a model the ability to act

KavinSchool

Why Tools?

An LLM on its own cannot do some things reliably:

- Do exact arithmetic on large numbers
- Look up current or private data
- Take an action in another system

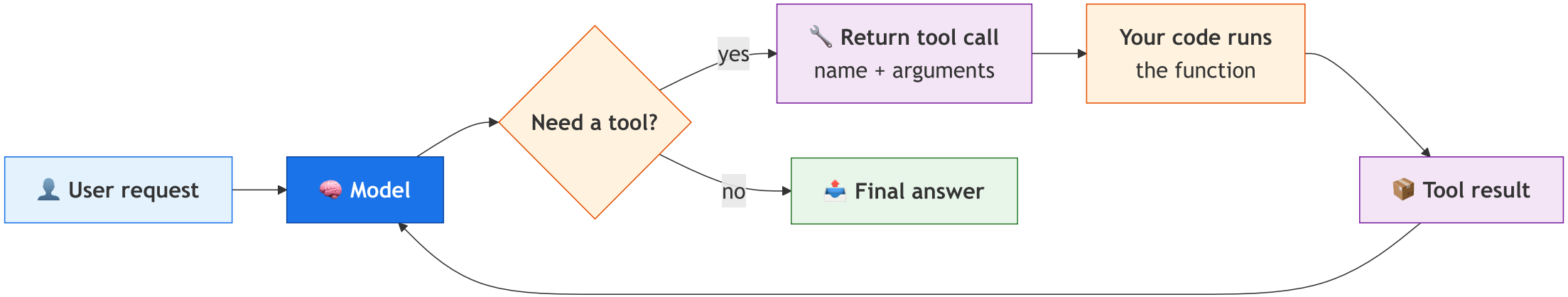
Tools let the model call your code to fill these gaps.

What Function Calling Is

You describe functions to the model. It decides when to call one and returns the name plus arguments. Your code runs the function and passes the result back.

- The model never runs code itself
- It only requests a call in a structured format
- You stay in control of what actually executes

The Function-Calling Loop



Describing a Tool

You give the model a name, a description, and a parameter schema.

```
def get_weather(city: str) -> str:
    """Return the current weather for a city."""
    ...
```

A clear name and docstring help the model decide when to call it and what to pass.

> **NOTE**
>
> The model reads the docstring and type hints to understand the tool. If the docstring is vague, the model will struggle to use the tool correctly.

Demo: The Function-Calling Loop

Run `day2/demos/demo-function-calling.py`.

- Expose two tools to the model with names, descriptions, and schemas
- Watch the model request a tool by name with structured arguments
- Your code runs the tool and passes the result back for the final reply

> Runs locally, no API key. The model requests a call; your code decides what runs.

Agentic AI

Multi-step reasoning and action

KavinSchool

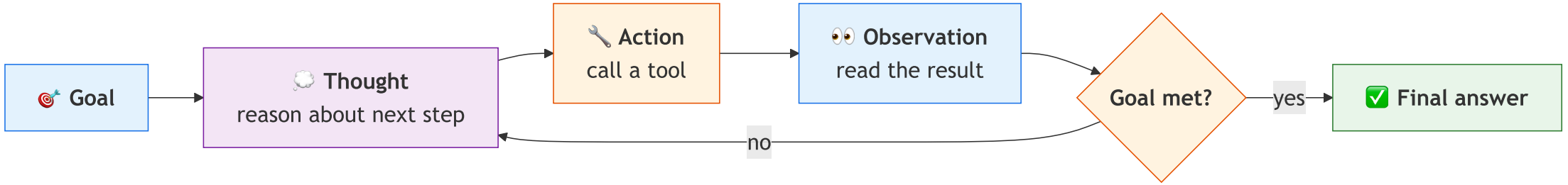
From Single Calls to Agents

A single function call handles one step. An agent chains many:

- Break a goal into steps
- Pick and call tools as needed
- Read results and decide what to do next
- Stop when the goal is met

The model drives the loop instead of you hard-coding each step.

The ReAct Pattern



ReAct (reasoning and acting) interleaves the two: think, act, observe, repeat.

In Practice: A customer service agent thinks "I need the user's order history," calls `get_orders(user_id)`, observes the result, then thinks "Now I need to check the return policy," before finally answering the user.

Why ReAct Works

- **Thought** makes the plan explicit and easier to debug
- **Action** uses tools for facts and computation
- **Observation** grounds the next step in real results

The loop keeps the model honest by checking against tool output each round.

Intuit Scenario: TurboTax Assistant

A TurboTax chat assistant uses tools to give accurate, personalized answers.

- **The challenge:** The LLM cannot do tax math accurately or know the user's W-2 tax form details.
- **The tools:** It has tools to fetch user documents, look up IRS (Internal Revenue Service) rules via MCP, and run calculator functions.
- **The agent:** Using ReAct, the model reasons: "I need to fetch the W-2 first (Action 1), then use the calculator to sum the wages (Action 2), before answering."

Demo: A ReAct Agent

Run `day2/demos/demo-react-agent.py`.

- Give the agent a goal it cannot answer in one step: tax on W-2 wages
- Watch the Thought, Action, Observation loop fetch the W-2, then calculate
- See each step grounded in real tool output before the final answer

> Runs locally, no API key. The loop keeps the model honest with real observations.

Agent Frameworks

Frameworks handle the loop, tool wiring, and parsing so you focus on the task.

Framework	Style
smolagents	Minimal, code-first agents
LangGraph	Graph of steps and state
LlamaIndex	Data and retrieval focused

Today's lab uses smolagents, with LangGraph as a stretch goal.

Keep Agents Safe

Agents take actions, so guardrails matter:

- Limit which tools an agent can use
- Cap the number of steps to avoid runaway loops
- Validate arguments before running a tool
- Log every action for review

Give an agent the least power it needs to do the job.

> **CAUTION**
>
> If a tool mutates data or takes an action (like sending an email), consider requiring human-in-the-loop approval before executing the tool call.

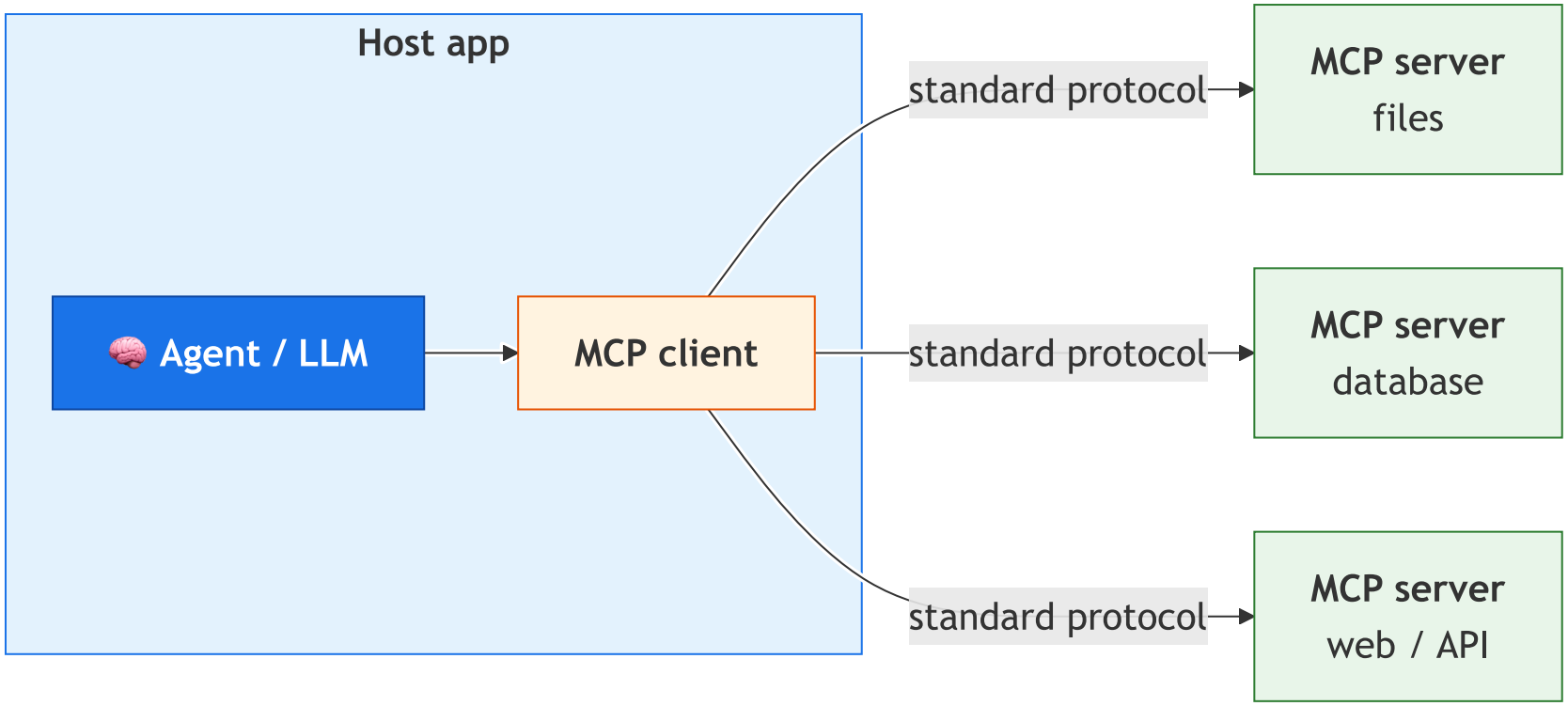
Model Context Protocol

A standard way to connect tools

KavinSchool

What MCP Solves

Without a standard, every tool needs custom wiring for every app. MCP defines one protocol so tools and models connect the same way.



How MCP Fits

- An **MCP server** exposes tools, data, or prompts
- An **MCP client** in your app connects to any server
- Write a tool once, use it from any MCP-aware agent

Think of it as a common plug for models and the systems they use.

> **TIP**
>
> You can use standard MCP servers (like GitHub, SQLite, or Slack) to instantly give your agents access to external systems without writing the API integration code yourself.

Breakout Lab 2.4

Function calling and a simple agent

KavinSchool

Lab 2.4: Build an Agent

Open `day2/lab2_4_agent/start.ipynb` in Colab.

- Write two small tools the model can call
- Build a smolagents CodeAgent that uses them
- Ask a question that needs a tool and watch the ReAct loop
- Trace each thought, action, and observation

> Stretch goal: rebuild the agent with LangGraph and compare the two styles.

Key Takeaways

- [x] Tools let a model act, look up data, and compute reliably
- [x] Function calling returns a structured request; your code runs it
- [x] Agents chain steps with the ReAct loop: think, act, observe
- [x] Frameworks like smolagents handle the loop so you focus on tools
- [x] MCP is a standard way to connect models to tools and data

Day 2 Wrap-up

What you built today

KavinSchool

Day 2 Recap

Today you went from language as data to working agents:

- [x] Embedded documents and ran similarity search
- [x] Compared BERT and GPT and saw the Transformer
- [x] Prompted for summarization, code, and labeling
- [x] Built a tool-using agent

Tomorrow: fine-tuning, RAG, and deployment.

Questions?

Next: Kahoot

Afternoon Kahoot: 10 minutes

KavinSchool